Lab 1 - Environment & Basic HTML / CSS

1. Course introduction and project orientation

a. Course Objectives

The Web Design and Development course equips students with the skills to build modern web interfaces (client-side frontend). Upon completion of this course, students will be able to:

- Accurately convert Figma designs into a responsive web interface.
- Handle interactive logic and dynamic data using JavaScript and the ReactJS library.
- Use AI tools (Vibe Coding) to accelerate programming and optimize source code.
- Package and manage source code using Git/GitHub and Deploy applications to the Internet.

b. Scoring method

- Attendance: 20%
- In-class exercise: 40%
- Homework: 40%

  Note:

  - All in-class assignments and homework must be completed individually..
  - Students should submit their assignments as Word files, including the GitHub link for the assignment (with access set to public) at the end of the file.
  - File naming: Lab<session name>_<Student ID>_<Student Name>.docx
    For example: *Lab1_26521234_Nguyễn Văn A.docx*

2. Setting up the development environment

a. Visual Studio Code (VS Code)

Download: https://code.visualstudio.com/download

- Windows: https://www.youtube.com/watch?v=wU7IQLIOwoo&t=22s
- MacOS: https://www.youtube.com/watch?v=_7PlHIkP2WQ

b. Git

Download: https://git-scm.com/install/

- Windows: https://www.youtube.com/watch?v=t2-l3WvWvqg
- MacOS: https://www.youtube.com/watch?v=FWDPqistXd4

c. Node.js

Download: https://nodejs.org/en/download

- Windows: https://www.youtube.com/watch?v=6vdQi_BK-uM
- MacOS: https://www.youtube.com/watch?v=0LWJ3gmScUY

d. Installation check successful

Open Terminal (on macOS/Linux) or Command Prompt/PowerShell (on Windows) and run the following commands one by one:

```
code --version
git --version
node --version
npm --version
```

Expected result: All four commands output the version number (e.g., *git version 2.4x.x*). If a *"Command not found"* error occurs, restart your computer or check the PATH environment variable.

3. Source Code Management with Git & GitHub

a. Sign up for a GitHub account

Sign up link: https://github.com/signup

b. Configuring Git identity on personal machine

Open Terminal / PowerShell and run these command (using your information):

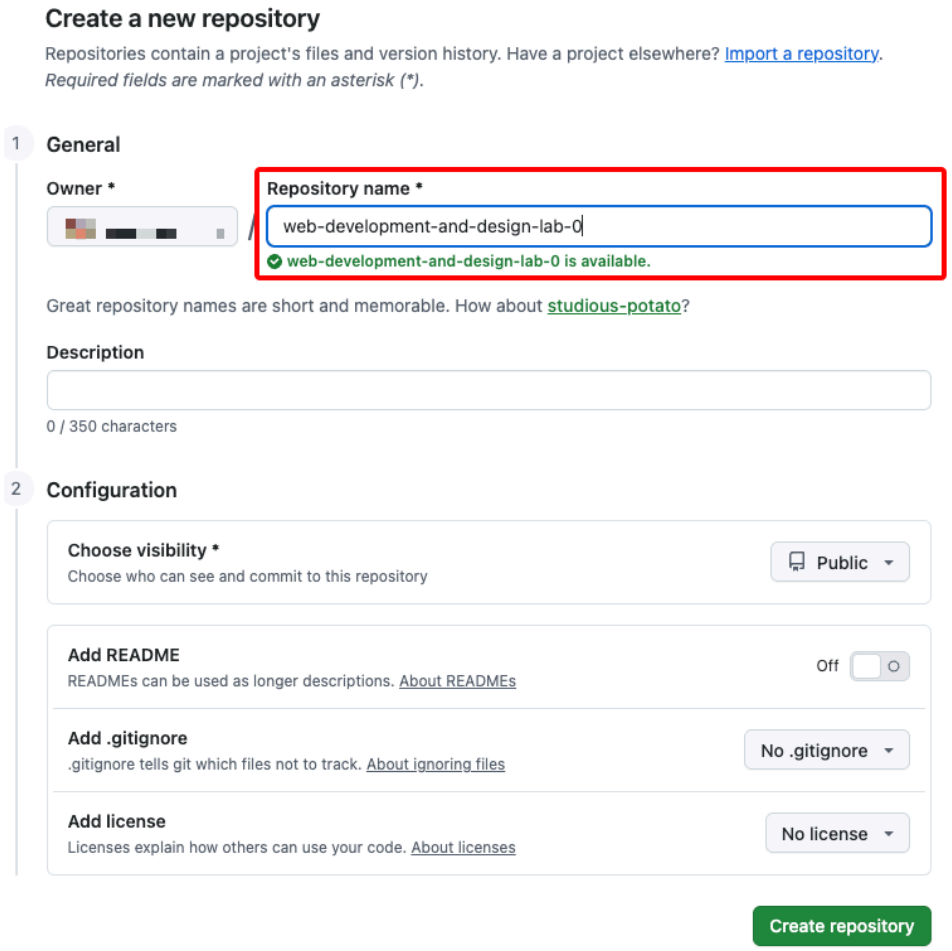
```
git config --global user.name "Nguyen Van A"
git config --global user.email "nguyenvana@gmail.com"
```

Recheck:

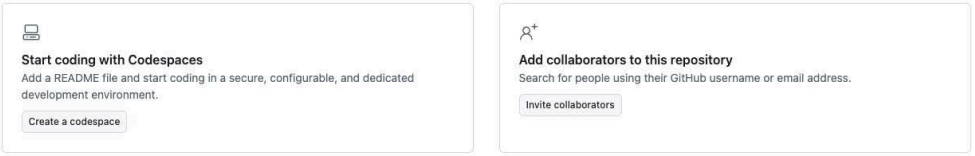
```
git config --list
```

c. Workflow with Git

- Create a new repository on GitHub.



- Initialize a local Git repository, connect it to a remote repository on GitHub, and push the code.



Quick setup — if you've done this kind of thing before

Set up in Desktop or HTTPS SSH https://github.com/nguyenn-12/web-development-and-design-lab-1.git

Get started by creating a new file or uploading an existing file. We recommend every repository include a README, LICENSE, and .gitignore.

...or create a new repository on the command line

```
echo "# web-development-and-design-lab-1" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/nguyenn-12/web-development-and-design-lab-1.git
git push -u origin main
```

...or push an existing repository from the command line

```
git remote add origin https://github.com/nguyenn-12/web-development-and-design-lab-1.git
git branch -M main
git push -u origin main
```

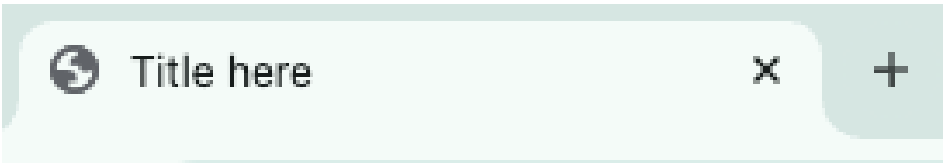
4. HTML & CSS

4.1. HTML

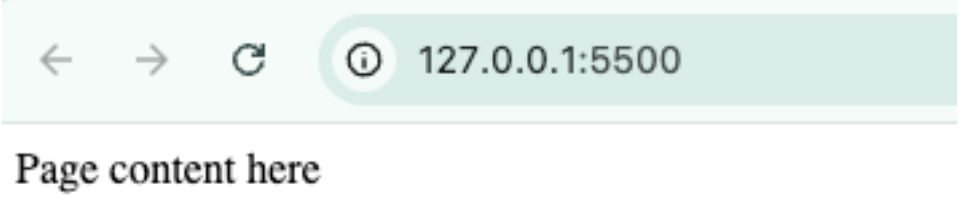
a. Minimal Structure:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Title here</title>
</head>
<body>
    Page content here
</body>
</html>
```

- <!DOCTYPE html>: Declares the document as using HTML5, enabling the browser to interpret and render the HTML according to modern standards.
- <html lang="en">: It is the root element that encloses all HTML content. `lang="en"` indicates that the primary language of the page is English.
- <head>: Contains page configuration information (metadata), such as character encoding, viewport, title, CSS, and other resources.
- <meta charset="UTF-8">: Declare the character encoding as UTF-8. This is necessary for the browser to correctly display characters, especially for languages with diacritics such as Vietnamese.
- <meta name="viewport" content="width=device-width, initial-scale=1.0">: It helps the website display correctly across various devices, especially mobile phones. `width=device-width` matches the page width to the screen width, while `initial-scale=1.0` sets the initial zoom level to 100%.
- <title>: Defines the webpage title, which is displayed on the browser tab. It also helps search engines understand the page's content.



- <body>: Contains all the content displayed on the website, such as headings, paragraphs, images, links, buttons, etc.



Exercise 1: Change the title, content, and icon of the website (suggestion: use `<link rel="icon" href="https://www.uit.edu.vn/media/Logo_UIT_In_bde69cac47.jpg">`)



Welcome to University of Information Technology!!!

b. Basic Element HTML

Group	Tag	Description
Document Structure	<html>	Root element that wraps all the HTML content on the page.
	<head>	Contains metadata that is not directly displayed, such as character encoding, viewport settings, title, and links to CSS/JS.
	<body>	Contains all the visible content of the webpage: headings, paragraphs, images, links, etc.
	<title>	Sets the title shown on the browser tab; also used by search engines.
	<meta>	Provides metadata such as character encoding, viewport, or SEO description.
	<link>	Links an external resource to the page, most commonly a CSS stylesheet or a favicon.
Headings & Paragraphs	<h1> - <h6>	Define headings from most important (h1) to least important (h6); used to structure content hierarchy.
	<p>	Defines a paragraph of text.
	 	Inserts a single line break without starting a new paragraph.
	<hr>	Draws a horizontal line, often used to separate sections of content.
Text Formatting (Inline)		Marks text as important; browsers render it bold by default.
		Marks text as emphasized; browsers render it italic by default.
	<b>	Makes text bold, with no extra semantic meaning.
	<i>	Makes text italic, with no extra semantic meaning.
		Generic inline container used to apply styling or scripting to a piece of text.
	<small>	Renders text in a smaller font, often used for fine print or disclaimers.
	<mark>	Highlights text, typically with a yellow background, to draw attention.

Group	Tag	Description
Lists		Creates an unordered (bulleted) list.
		Creates an ordered (numbered) list.
		Defines each item inside a or .
	<dl>	Creates a description list, grouping terms with their definitions.
	<dt>	Defines a term inside a description list.
	<dd>	Defines the description/explanation of a term inside a description list.
Links & Media	<a>	Creates a hyperlink to another page, section, or resource.
	<img>	Embeds an image into the page.
	<video>	Embeds a video with playback controls.
	<audio>	Embeds an audio file with playback controls.
	<iframe>	Embeds another HTML page or external content inside the current page.
Tables	<table>	Defines a table for displaying tabular data.
	<tr>	Defines a row within a table.
	<th>	Defines a header cell within a table row.
	<td>	Defines a standard data cell within a table row.
	<thead>	Groups the header row(s) of a table.
	<tbody>	Groups the main body rows of a table.
Forms & Input	<form>	Wraps a group of input controls and defines how/where the data is submitted.
	<input>	Creates a single input field (text, checkbox, radio, etc.), depending on the type attribute.
	<label>	Provides a text description for a form control, improving accessibility and usability.
	<button>	Creates a clickable button, often used to submit a form or trigger a script action.
	<select>	Creates a dropdown list of options.
	<textarea>	Creates a multi-line text input field.
Semantic Layout	<header>	Defines the introductory section of a page or section, typically containing a logo or navigation.
	<nav>	Defines a block of navigation links.
	<main>	Defines the main, unique content of the page.
	<section>	Defines a thematic grouping of content, usually with a heading.
	<article>	Defines a self-contained piece of content that could stand alone, e.g. a blog post.
	<aside>	Defines content indirectly related to the main content, such as a sidebar.
	<footer>	Defines the footer section of a page or section, typically containing copyright or contact info.
		Generic block-level container with no semantic meaning, used for grouping and styling.

Important Notes

- **Block-level elements** start on a new line and take up the full width available (e.g. , <p>, <h1>-<h6>, <ul>, <table>).
- **Inline elements** flow within a line of text and only take up as much width as their content needs (e.g. , <a>, , , <img>).
- **Self-closing (void) elements** have no closing tag and no content, e.g.
, <hr>, <img>, <meta>, <input>, <link>.
- **Nesting rule** block-level elements can contain inline elements, but inline elements should not contain block-level ones.

Example 1:

Course Objectives

This is a **simple demo page** using the *basic HTML elements* from Lab 1. It shows headings, lists, tables, links, media, and forms all in one file.

What you'll learn

- Convert Figma designs into a responsive interface
- Handle dynamic data with JavaScript and React
- Use AI tools to speed up coding
- Manage source code with Git and deploy it online

```
<h2>Course Objectives</h2>
<p>This is a <strong>simple demo page</strong> using the <em>basic HTML elements</em>
  from Lab 1. It shows <mark>headings</mark>, <mark>lists</mark>, <mark>tables</mark>,
  <mark>links</mark>, <mark>media</mark>, and <mark>forms</mark> all in one file.</p>

<h3>What you'll learn</h3>
<ul>
  <li>Convert Figma designs into a responsive interface</li>
  <li>Handle dynamic data with JavaScript and React</li>
  <li>Use AI tools to speed up coding</li>
  <li>Manage source code with Git and deploy it online</li>
</ul>
```

Example 2:

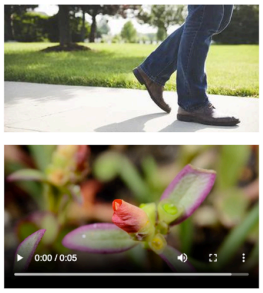
```
<h2>Scoring Method</h2>
<table border="1" cellpadding="8" cellspacing="0">
  <thead>
    <tr>
      <th>Component</th>
      <th>Weight</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Attendance</td>
      <td>20%</td>
    </tr>
    <tr>
      <td>In-class exercise</td>
      <td>30%</td>
    </tr>
    <tr>
      <td>Homework</td>
      <td>25%</td>
    </tr>
    <tr>
      <td>Project</td>
      <td>25%</td>
    </tr>
  </tbody>
</table>
```

Scoring Method

Component	Weight
Attendance	20%
In-class exercise	30%
Homework	25%
Project	25%

Example 3:

Media & Links



Visit uit.edu.vn for more information.

```
<h2>Media &amp; Links</h2>

<p><img src="https://picsum.photos/400/200" alt="Sample placeholder image" width="400"></p>

<p>
  <video width="400" controls>
    <source src="https://interactive-examples.mdn.mozilla.net/media/cc0-videos/flower.mp4" type="video/mp4">
    Your browser does not support the video tag.
  </video>
</p>

<p><small>Visit <a href="https://www.uit.edu.vn" target="_blank">uit.edu.vn</a> for more information.</small></p>
```

Example 4:

Join the Club

Full name:
Nguyen Van A

Email:
you@gm.uit.edu.vn

Role:
Member

Message:

Send me updates

Submit

```
<h2>Join the Club</h2>
<form action="#" method="post">
  <label for="name">Full name:</label><br>
  <input type="text" id="name" name="name" placeholder="Nguyen Van A"><br><br>

  <label for="email">Email:</label><br>
  <input type="email" id="email" name="email" placeholder="you@gm.uit.edu.vn"><br><br>

  <label for="role">Role:</label><br>
  <select id="role" name="role">
    <option value="member">Member</option>
    <option value="mentor">Mentor</option>
  </select><br><br>

  <label for="message">Message:</label><br>
  <textarea id="message" name="message" rows="3" cols="30"></textarea><br><br>

  <input type="checkbox" id="updates" name="updates">
  <label for="updates">Send me updates</label><br><br>

  <button type="submit">Submit</button>
</form>
```

Exercise 2:

Use the VS Code Emmet shortcut ! (Exclamation mark) + Enter to generate the minimal HTML5 boilerplate. Then, inside the <body> tag, build a complete *semantic personal CV/resume* structure, freely designed with your own content and layout.

Requirements:

- Organize the page using semantic landmarks such as <header>, <nav>, <main>, <section>, <article>, <aside>, and <footer>.
- Include at least: an introduction (name, title, photo), education/experience, skills, and a contact section.
- Reuse the basic elements from this lesson where appropriate, e.g. headings, lists, table, links, and images.

Practice:

- Right-click index.html and choose *"Open with Live Server"* to preview the page in your browser.
- Open DevTools (F12) → *Elements* tab to see how the browser parses your semantic structure tree.

4.2. CSS

a. Adding CSS to Your HTML

There are three ways to add CSS to an HTML page:

- **Inline CSS** – written directly inside an element's `style="..."` attribute. Only affects that one element.

Web Design & Development

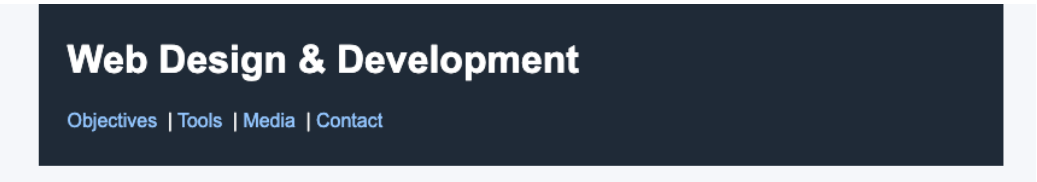
```
<h1 style="color: rgb(47, 181, 78); text-align: center;">
  Web Design &amp; Development
</h1>
```

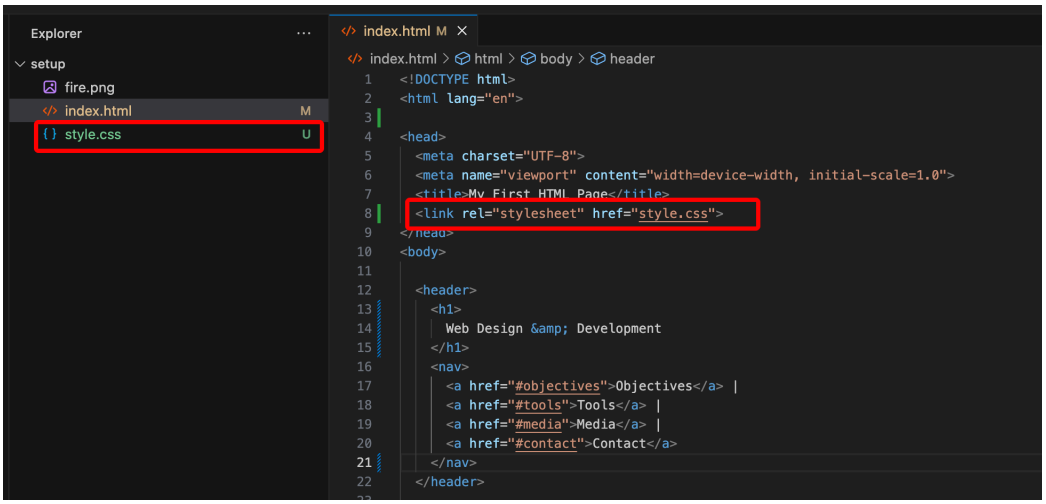
- **Internal CSS** – written inside a `<style>` block in the `<head>`. Affects the whole page.

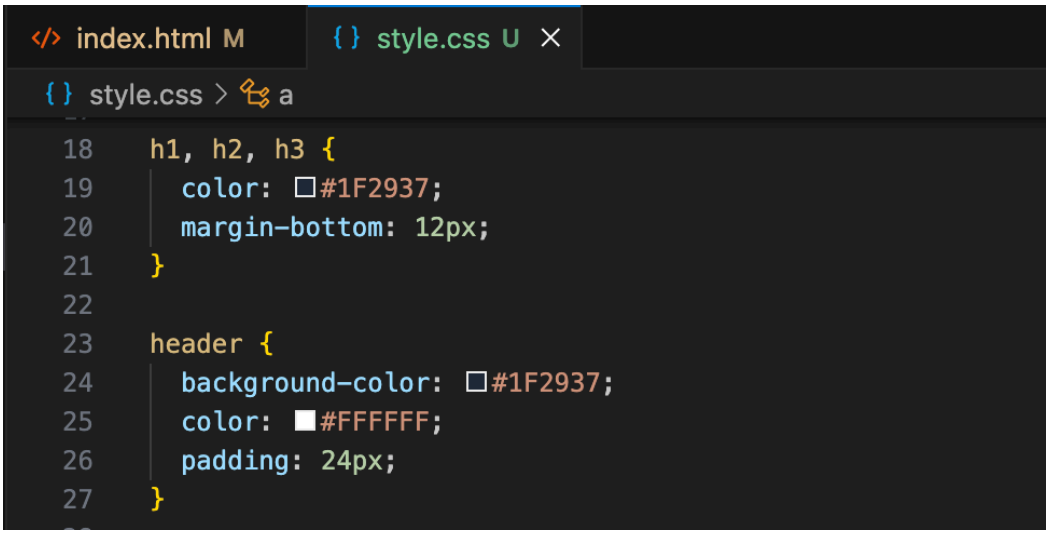
Web Design & Development

```
index.html M
index.html > html > head
1  <!DOCTYPE html>
2  <html lang="en">
3
4  <style>
5    h1 {
6      color: rgb(235, 18, 18);
7      text-align: center;
8    }
9  </style>
10
11 <head>
12   <meta charset="UTF-8">
13   <meta name="viewport" content="width=device-width, initial-scale=1.0">
14   <title>My First HTML Page</title>
15 </head>
16 <body>
17
18   <header>
19     <h1>
20       Web Design &amp; Development
21     </h1>
22     <nav>
23       <a href="#objectives">Objectives</a> |
24       <a href="#tools">Tools</a> |
25       <a href="#media">Media</a> |
26       <a href="#contact">Contact</a>
27     </nav>
28   </header>
```

- **External CSS** (recommended) – written in a separate `.css` file and linked with `<link rel="stylesheet" href="style.css">` inside the `<head>`.







b. CSS Syntax and Selectors

Every CSS rule follows the same pattern:

`selector { property: value; }`

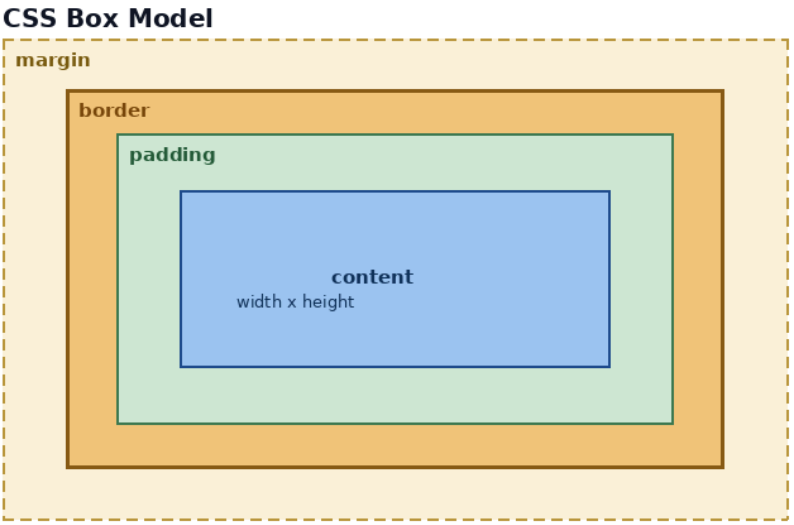
The three most common selector types:

Selector	Syntax	Example
Element	`tagName { }`	p { color: navy; } styles every <p>
Class	`.className { }`	reusable on many tags: <button class="btn">
ID	`#idName { }`	targets exactly one element: <section id="contact">

Note: a class can be reused many times per page; an `id` should only be used once.

c. The Box Model

Every HTML element is a rectangular box made of four layers, from the inside out: content, padding, border, and margin.

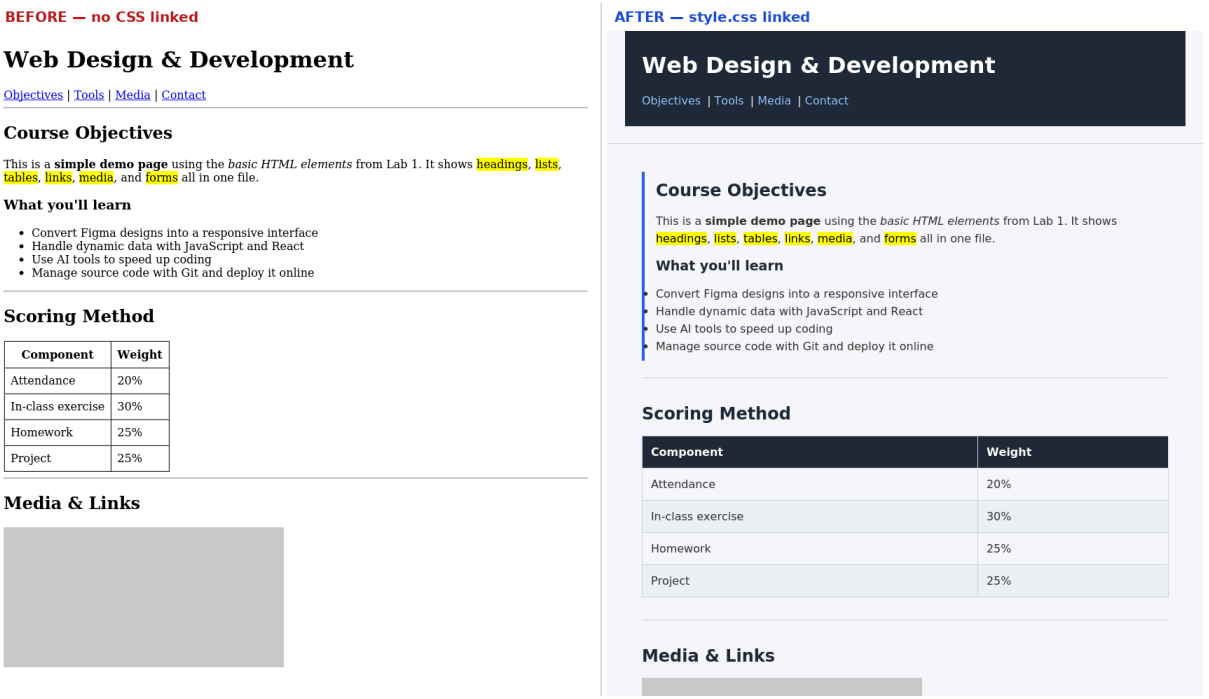


d. Common CSS Properties

Category	Property	Description
Typography	color, font-size, font-family, font-weight	Text color, size, typeface, and boldness.
Box model	width, height, margin, padding, border	Size of an element and the space around/inside it.
Background	background-color, background-image	Fill color or image behind an element.
Display	display, position	How an element is laid out on the page (block, flex, grid...).
Text align	text-align, text-decoration	Horizontal alignment and underline/strike styling.
Effects	border-radius, box-shadow, cursor	Rounded corners, shadows, and pointer style on hover.

e. Example: Styling index.html

Below is the same index.html page from the earlier exercise, before and after linking an external style.css file:



Exercise 3:

Create a style.css file and link it to the personal CV page you built in Exercise 3. Style the page using the CSS concepts covered in this lesson.

Exercise 4:

Now that your CV is styled, bring it to life with a few small interactions. Add **hover** and **transition** effects so the page feels more polished and responsive to the user.

Requirements:

- Add a hover effect on at least one interactive element (e.g. nav links, buttons, or your profile photo).
- Use the transition property so hover changes animate smoothly instead of snapping instantly.
- (Bonus) Add a subtle transform – like scale() or translateY() – on hover for extra polish.

Reference Snippets:

1. Nav link color transition

```
nav a {
  color: #333333;
  text-decoration: none;
  transition: color 0.3s ease;
}

nav a:hover {
  color: #2563EB;
}
```

2. Button lift with scale transform

```
.btn {
  transition: transform 0.2s ease, background-color 0.2s ease;
}

.btn:hover {
  transform: scale(1.05);
  background-color: #1D4ED8;
}
```

3. Card hover with shadow lift

```
.card {
  transition: box-shadow 0.3s ease, transform 0.3s ease;
}

.card:hover {
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.15);
  transform: translateY(-4px);
}
```

Tip: keep transitions short (0.2s–0.3s) – fast, subtle motion feels responsive; long animations feel sluggish.

5. Deploy to Github Pages

Once your CV page is ready, publish it to the web using GitHub Pages.

Step 1: Commit and Push Your Code

From your project folder, run:

```
git add .
git commit -m "Add personal CV with CSS styling"
git push origin main
```

Step 2: Enable GitHub Pages

- Open your repository on GitHub and go to **Settings** → **Pages** (left sidebar).
- Under *"Build and deployment"*, set **Source** to *"Deploy from a branch"*.
- Select the main branch and the /(root) folder, then click **Save**.
- Wait 1–2 minutes and refresh the page – GitHub will display your live URL: https://username.github.io/repo-name/.

Common Errors & Debugging

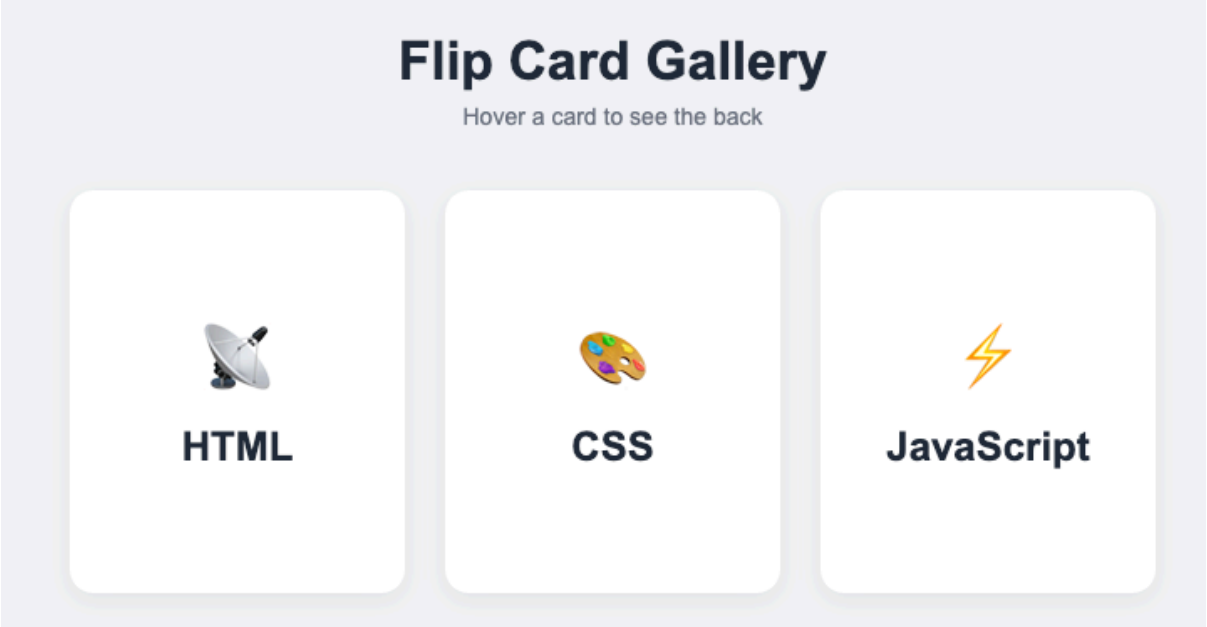
- **404 Page Not Found** – make sure index.html sits at the root of the selected branch/folder (not inside a subfolder), and that the filename matches exactly – GitHub Pages URLs are case-sensitive.
- **Page loads but the CSS is missing** – check that <link href="style.css"> uses a relative path, not a local absolute path like C:\Users\...; confirm the file was actually pushed with git status.
- **"remote: Repository not found"** on push – check the remote URL with git remote -v and confirm you're logged into the correct GitHub account.
- **Changes not showing after deploy** – GitHub Pages can cache the old version; try a hard refresh (Ctrl+Shift+R) or wait a few minutes.
- **Push rejected / merge conflict** – pull the latest changes first with git pull --rebase before pushing again.

6. Homework

Complete **all** exercises below using HTML and CSS. Your grade will be proportional to how closely your result matches the reference screenshot.

1. Flip Card Gallery

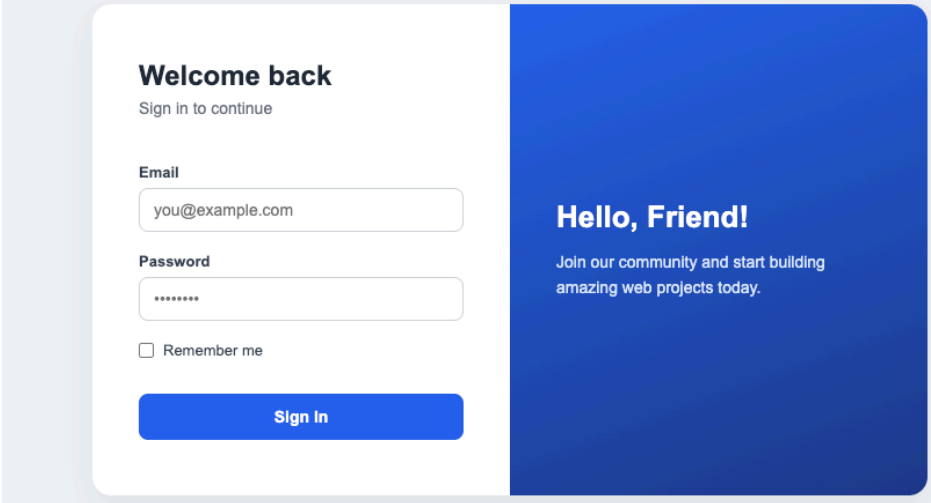
3 cards that flip in 3D on hover, showing a title on the front and a short description on the back.



Focus: perspective, rotateY, backface-visibility, smooth transition.

2. Login / Sign-up Card

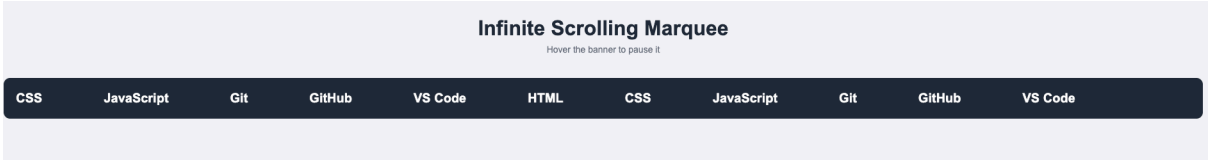
A centered two-column card: a login form on one side, a colored decorative panel on the other.



Focus: Flexbox/Grid centering, form styling, border-radius, box-shadow.

3. Infinite Scrolling Marquee

A banner where a row of text/logos scrolls continuously and loops seamlessly, pausing on hover.



Focus: @keyframes, animation-play-state, infinite looping.